

CLIENT PROJECT DOCUMENTATION

AcademyOps

Lead-to-Enrollment Management System

Software Requirements & Implementation Specification

Document	Software Requirements & Implementation Specification (SRS)
Project	AcademyOps — Lead-to-Enrollment Management System
Client	EasySkill Career Academy (ECA)
Engagement type	Hands-on training engagement (industry simulation)
Version	1.0
Date	30 May 2026
Status	Issued for Implementation

Table of Contents

Program Overview

1.1 Purpose of this Document

This specification packages a progressive, multi-module software project as a simulated client engagement. Each module is written as an industry-style work package so that the implementing engineer experiences realistic requirements, planning, and acceptance practices — the same shape of documentation used on large-scale company projects.

1.2 Client & Product Context

Client: EasySkill Career Academy (ECA). Product: AcademyOps, a lead-to-enrollment management system that captures leads from marketing channels, tracks them through a defined sales pipeline, supports counselor follow-up, and provides analytics on funnel performance. The product is built up module by module, each delivering a working slice of the final system.

1.3 Sales Pipeline Definition

Leads progress through the following stages: New → Contacted → Qualified → Demo → Enrolled, with Lost as a terminal state reachable from any active stage. This pipeline is the shared domain model across all modules.

1.4 Engagement Model & Sequencing

- Modules are delivered sequentially; each depends on the ones before it.
- Each work package is treated as a deliverable with its own acceptance review against its evaluation criteria.
- A short build log is maintained per module: what was built, what was difficult, and which decisions were taken.
- Definition of Done for every module: functional requirements met, testing checklist passed, deliverables produced, and code committed.

1.5 Global Engineering Standards

- Version control: commit frequently with clear messages; never commit secrets.
- Configuration: from WP-07 onward, secrets and configuration are supplied via the environment and never hard-coded; earlier modules avoid hard-coded paths where practical.
- Code quality: clear naming, separation of concerns, and no dead code.
- Documentation: every module updates the README where relevant.

1.6 Module Roadmap Summary

WP	Module	New capability introduced	Est. effort
00	Project Initiation & Environment Setup	Git, project structure	~0.5 day
01	Lead Data Model & Persistence Layer	SQLite, Repository pattern	~1.5 days
02	Data Ingestion & Cleansing Pipeline	Real-world data validation	~1.5 days
03	Lead Management REST API (Flask)	REST API design	~1–1.5 days
04	Automated Test Suite & Quality Gates	pytest, CI	~1.5 days

WP	Module	New capability introduced	Est. effort
05	Funnel Analytics & Statistical Reporting	Applied statistics	~1–1.5 days
06	Operations Dashboard (Streamlit)	Dashboards	~1–1.5 days
07	Production API Re-platform	FastAPI, PostgreSQL, Pydantic	~2 days
08	Lead Intent Classification & Automation	Intent classification / LLM	~1 day
09	System Integration & Release (Capstone)	Integration, deployment	~1.5 days

1.7 Glossary

- Lead: a prospective student captured from a marketing channel.
- Stage: the lead's current position in the sales pipeline.
- Funnel: the sequence of stages and the drop-off between them.
- Conversion rate: the proportion of leads that progress from one stage to the next (or to Enrolled).
- Repository pattern: a class that encapsulates all data access for an entity, isolating SQL from the rest of the app.
- Quarantine: a separate file where rejected import rows are stored with their rejection reason.
- CRUD: Create, Read, Update, Delete operations.
- REST / Endpoint: an HTTP-based interface; an endpoint is a single addressable operation on it.
- Pydantic: a Python library used by FastAPI to validate and type request/response data.
- CI: continuous integration — automated running of the test suite on each change.

WP-00 — Project Initiation & Environment Setup

1. Project Overview & Objective

Establish the version-controlled codebase, reproducible environment, and project structure that every later module builds on. This is the foundation work that, in a real engagement, must be completed and approved before any feature development begins.

Objective: produce a repository that any engineer can clone and have running within minutes by following the README alone.

2. Business Requirements

- **BR-1:** The client must be able to onboard a new engineer to the codebase quickly and without tribal knowledge.
- **BR-2:** All work must be auditable through version history.
- **BR-3:** The project must run consistently across different developer machines.

3. Functional Requirements

- **FR-1:** A Git repository hosted remotely (GitHub) with a clear default branch.
- **FR-2:** A documented folder structure separating source, tests, data, and scripts.
- **FR-3:** A pinned dependency manifest (requirements.txt) installed inside an isolated virtual environment.
- **FR-4:** A README describing the project purpose, setup steps, and run instructions.
- **FR-5:** A .gitignore that excludes secrets, virtual environments, caches, and local database files.

4. Expected Output

A GitHub repository containing the agreed folder structure, README, dependency manifest, and .gitignore, with an initial commit. A fresh clone followed by the documented setup steps completes without errors.

5. Step-by-Step Implementation Plan

1. Create the remote repository and clone it locally.
2. Initialise the folder structure (src/, tests/, data/, scripts/).
3. Create and activate a virtual environment; record the Python version used.
4. Add .gitignore and an initial requirements.txt.
5. Write the initial README covering purpose, setup, and how to run.
6. Make the initial commit, push, and verify a fresh clone sets up cleanly.

6. Estimated Time Required

Phase	Effort
Planning & repository creation	0.5 h
Structure & environment setup	1.5 h
README authoring	1.0 h
Clean-clone verification	0.5 h

Estimated total: ≈ 3.5 hours (about half a day)

7. Technologies to be Used

Python 3.11+, Git

8. Tools to be Used

GitHub, VS Code (or preferred IDE), Terminal, Python venv

9. Database Requirements

Not applicable at this stage.

10. API Requirements

Not applicable at this stage.

11. UI/UX Expectations

Not applicable — the deliverable is a repository. README readability is the only “interface” requirement.

12. Testing Checklist

- A fresh clone plus the documented steps produces a runnable environment.
- .gitignore prevents committing the virtual environment, caches, secrets, and .db files.
- requirements.txt installs cleanly in a brand-new environment.
- README clearly answers: what the project is, how to set it up, and how to run it.

13. Deliverables

- GitHub repository URL.
- README.md, requirements.txt, and .gitignore.
- An initial commit visible in the history.

14. Evaluation Criteria

Criterion	Weight
Repository hygiene — clean structure, no stray or committed artifacts	30%
Reproducibility — a fresh clone runs by following the README	40%
Documentation clarity — README is complete and unambiguous	30%

WP-01 — Lead Data Model & Persistence Layer

1. Project Overview & Objective

Implement the canonical Lead entity and a reusable persistence layer (Repository pattern) over SQLite, with safe parameterised queries, transactions, structured domain errors, and logging. This module becomes the system of record that every later module depends on.

2. Business Requirements

- **BR-1:** The academy must store and retrieve lead records reliably.
- **BR-2:** A lead's progress through the sales pipeline must be tracked accurately.
- **BR-3:** Data operations must be safe against corruption and injection.
- **BR-4:** Operational issues must be diagnosable after they occur.

3. Functional Requirements

- **FR-1:** A Lead entity with id, name, phone, source, stage, notes, created_at, and updated_at.
- **FR-2:** Pipeline stage constrained to one of: New, Contacted, Qualified, Demo, Enrolled, Lost.
- **FR-3:** Create, read, list, update-stage, and delete operations exposed through a LeadRepository class.
- **FR-4:** All SQL executed through parameterised statements.
- **FR-5:** Multi-statement writes executed inside a transaction with commit/rollback.
- **FR-6:** Domain errors (for example, LeadNotFound) raised as custom exceptions rather than returning None.
- **FR-7:** All create/update/delete operations and all errors written to a log file.
- **FR-8:** A command-line tool to add, list, and update leads.

4. Expected Output

A working command-line tool backed by a SQLite database. Data persists across runs, invalid operations raise clear domain errors instead of crashing, and an application log records activity.

5. Step-by-Step Implementation Plan

1. Define the database schema and create an initialisation routine.
2. Implement the Lead model and the stage enumeration.
3. Build LeadRepository with create, get, list, update_stage, and delete methods.
4. Apply parameterised queries and transaction handling throughout.
5. Define and raise custom exceptions for domain error cases.
6. Configure file-based logging across all operations.
7. Build the CLI wrapper over the repository.
8. Manually exercise every command and confirm data persists.

6. Estimated Time Required

Phase	Effort
Schema & model	2.0 h
Repository implementation	3.0 h
Errors & logging	1.5 h
CLI wrapper	2.0 h

Phase	Effort
Manual verification	1.5 h

Estimated total: $\approx$ 10 hours (about 1.5 days)

7. Technologies to be Used

Python 3.11+, SQLite3, Python logging module

8. Tools to be Used

VS Code, DB Browser for SQLite, Git

9. Database Requirements

- A single SQLite database file (for example, academyops.db), which must not be committed.
- leads table: id (INTEGER PK, autoincrement), name (TEXT, required), phone (TEXT, required, unique), source (TEXT), stage (TEXT, constrained), notes (TEXT), created_at (TEXT/ISO), updated_at (TEXT/ISO).
- Indexes on phone and on stage to support lookups and filtering.

10. API Requirements

Not applicable — this module provides a library and CLI only.

11. UI/UX Expectations

- Command-line interface only.
- Clear prompts, a readable tabular listing of leads, and helpful error messages.
- No graphical user interface.

12. Testing Checklist

- Creating a lead persists it; restarting the CLI still shows it.
- Updating a stage is reflected in storage and refreshes updated_at.
- Deleting a non-existent lead raises LeadNotFound rather than crashing.
- A duplicate phone number is rejected.
- An injection attempt passed as input has no effect (parameterisation holds).
- The log file records each create/update/delete and every error.

13. Deliverables

- Model and repository module(s).
- CLI entry point.
- Schema/initialisation script.
- A sample log file demonstrating recorded activity.

14. Evaluation Criteria

Criterion	Weight
Correctness — CRUD works and data persists	30%
Safety — parameterised queries, transactions, no crashes on bad input	25%
Code design — clean repository abstraction and separation of concerns	25%
Observability — meaningful logging and clear exceptions	20%

WP-02 — Data Ingestion & Cleansing Pipeline

1. Project Overview & Objective

Build a robust importer that loads a messy external CSV of leads into the system of record, cleaning, validating, and de-duplicating the data while quarantining bad rows and producing a reconciliation report. This mirrors the real task of ingesting leads exported from ad platforms and spreadsheets.

2. Business Requirements

- **BR-1:** Leads arrive from external sources in inconsistent formats.
- **BR-2:** Bad or duplicate data must never pollute the system of record.
- **BR-3:** The team must be able to account for every row in an import.

3. Functional Requirements

- **FR-1:** Accept a CSV file with the expected set of columns.
- **FR-2:** Normalise values — trim whitespace, standardise casing, and map source aliases to canonical names.
- **FR-3:** Validate required fields and basic formats (for example, a plausible phone number).
- **FR-4:** De-duplicate by phone, both within the incoming batch and against existing records.
- **FR-5:** Load valid rows into the database via the WP-01 repository.
- **FR-6:** Write rejected rows, with their rejection reasons, to a separate quarantine file.
- **FR-7:** Emit a summary report: total / imported / skipped / de-duplicated.

4. Expected Output

Running the importer on a messy file loads only clean, unique records, produces a quarantine file of rejected rows with reasons, and prints (or writes) a reconciliation summary.

5. Step-by-Step Implementation Plan

1. Prepare a representative messy CSV (200+ rows with deliberately seeded defects).
2. Implement CSV reading and column mapping.
3. Implement the normalisation and validation rules.
4. Implement de-duplication logic.
5. Load valid rows through the repository inside a transaction.
6. Write rejected rows and reasons to the quarantine file.
7. Produce the reconciliation summary.
8. Verify counts reconcile: imported + skipped + de-duplicated equals total.

6. Estimated Time Required

Phase	Effort
Test data preparation	1.0 h
Read & column mapping	1.5 h
Clean & validate	2.5 h
De-duplication	1.5 h
Load & reporting	2.0 h
Verification	1.5 h

Estimated total: ≈ 10 hours (about 1.5 days)

7. Technologies to be Used

Python, csv / pandas, SQLite3 (via the WP-01 repository)

8. Tools to be Used

VS Code, DB Browser for SQLite, Faker (optional, for test data), Git

9. Database Requirements

- Reuses the WP-01 schema.
- The importer must respect the phone uniqueness constraint.

10. API Requirements

Not applicable.

11. UI/UX Expectations

- Command-line invocation with arguments (input path and optional output paths).
- A clear console summary at the end of the run.

12. Testing Checklist

- A clean file imports 100% of its rows.
- Rows missing required fields are quarantined with a stated reason.
- Duplicate phone numbers are collapsed or rejected, never inserted twice.
- Source aliases normalise to canonical values.
- Summary counts reconcile exactly with the input row count.
- Re-running the same file does not create duplicate records.

13. Deliverables

- The importer script.
- A sample messy input CSV.
- An example quarantine file and summary report.

14. Evaluation Criteria

Criterion	Weight
Data quality — output is clean, unique, and valid	35%
Robustness — no crashes on malformed rows; every row is accounted for	30%
Reporting — an accurate, reconciling summary	20%
Reusability — the importer uses the repository rather than raw SQL	15%

WP-03 — Lead Management REST API (Flask)

1. Project Overview & Objective

Expose the lead system over a versioned HTTP REST API so downstream clients (dashboards, bots) can integrate without touching the database directly. The API must support filtering, pagination, input validation, and consistent error responses.

2. Business Requirements

- **BR-1:** Multiple internal tools need programmatic access to lead data.
- **BR-2:** Integrations must not depend on direct database access.
- **BR-3:** The API must behave predictably for the developers who consume it.

3. Functional Requirements

- **FR-1:** List leads with filtering by stage and by source.
- **FR-2:** Paginate list results using page and limit parameters.
- **FR-3:** Retrieve a single lead by id.
- **FR-4:** Create a lead.
- **FR-5:** Update a lead's stage.
- **FR-6:** Delete a lead.
- **FR-7:** Validate request bodies and reject invalid input with status 400 and a clear message.
- **FR-8:** Return appropriate status codes and JSON error bodies through a global error handler.

4. Expected Output

A running Flask service exercisable with curl or Postman that returns correct status codes and JSON, handling invalid input and missing resources gracefully.

5. Step-by-Step Implementation Plan

1. Scaffold the Flask application and wire it to the WP-01 repository.
2. Implement the list endpoint with filtering and pagination.
3. Implement the retrieve, create, update-stage, and delete endpoints.
4. Add request validation and input parsing.
5. Register global error handlers that return JSON.
6. Document the endpoints (in the README or a short API reference).
7. Verify all endpoints and their status codes manually.

6. Estimated Time Required

Phase	Effort
Scaffold & wiring	1.0 h
Endpoints	3.5 h
Validation & error handling	2.0 h
API documentation	1.0 h
Verification	1.5 h

Estimated total: ≈ 9 hours (about 1 to 1.5 days)

7. Technologies to be Used

Python, Flask, SQLite (via the repository), JSON

8. Tools to be Used

Postman or curl, VS Code, Git

9. Database Requirements

- Reuses the WP-01 schema.
- No new tables are introduced.

10. API Requirements

Method	Endpoint	Purpose	Status codes
GET	/api/v1/leads	List leads; supports stage, source, page, limit query params	200
GET	/api/v1/leads/{id}	Retrieve a single lead	200 / 404
POST	/api/v1/leads	Create a lead	201 / 400
PATCH	/api/v1/leads/{id}/stage	Move a lead to a new stage	200 / 400 / 404
DELETE	/api/v1/leads/{id}	Delete a lead	204 / 404

Notes: Conventions: JSON request and response bodies; errors returned as {"error": "..."}; field names consistent across endpoints.

11. UI/UX Expectations

- No graphical UI — the API contract is the interface.
- Responses must be clean JSON with stable, consistent field names.

12. Testing Checklist

- Each endpoint returns its documented status code.
- Invalid JSON or a missing required field returns 400 with a clear message.
- An unknown id returns 404, never 500.
- Pagination returns the correct slice and reports the total count.
- Filtering by stage and by source works and the two combine correctly.
- No stack traces leak to clients.

13. Deliverables

- The Flask application code.
- An API reference (endpoints, parameters, sample requests and responses).
- A Postman collection or curl script.

14. Evaluation Criteria

Criterion	Weight
API correctness — endpoints behave per the contract	30%
Error handling — correct codes, no leaked stack traces	25%
Contract clarity — documented and consistent	20%
Code quality — clear separation between web layer and data layer	15%
Usability — easy to call and understand	10%

WP-04 — Automated Test Suite & Quality Gates

1. Project Overview & Objective

Establish automated tests for the data layer and the API so the system can be changed safely. Tests must run against isolated data, and ideally a continuous-integration job runs them on every push. This is the single biggest step toward company-grade engineering discipline.

2. Business Requirements

- **BR-1:** The client must be able to evolve the product without introducing regressions.
- **BR-2:** Releases must be backed by repeatable, automated verification.

3. Functional Requirements

- **FR-1:** Unit tests for the repository covering CRUD, error cases, and constraints.
- **FR-2:** API tests covering happy paths and edge cases.
- **FR-3:** Tests run against an isolated (temporary or in-memory) database.
- **FR-4:** Tests are independent and order-agnostic.
- **FR-5:** (Optional) A CI job runs the suite on every push.

4. Expected Output

A green test suite runnable with a single command, covering the core logic and key edge cases, with no dependence on real data.

5. Step-by-Step Implementation Plan

1. Add pytest and configure the test layout.
2. Create fixtures that provide an isolated database per test.
3. Write repository unit tests, including LeadNotFound and duplicate handling.
4. Write API tests for status codes, validation, pagination, and filtering.
5. Ensure tests are isolated and independent of one another.
6. (Optional) Add a GitHub Actions workflow that runs the suite.

6. Estimated Time Required

Phase	Effort
Setup & fixtures	2.0 h
Repository tests	2.5 h
API tests	3.0 h
CI workflow (optional)	1.0 h
Cleanup & review	1.0 h

Estimated total: ≈ 9.5 hours (about 1.5 days)

7. Technologies to be Used

Python, pytest, Flask test client

8. Tools to be Used

pytest, GitHub Actions (optional), coverage (optional), Git

9. Database Requirements

- A dedicated test database (temporary file or in-memory).
- The development or production database is never used by tests.

10. API Requirements

Tests target the WP-03 API; no new endpoints are added.

11. UI/UX Expectations

Not applicable — test reports and CLI output are the only surface.

12. Testing Checklist

- Happy-path tests exist for each repository method and each endpoint.
- Edge cases are covered: not found, duplicate, invalid input, and empty results.
- Each test sets up and tears down its own data.
- The suite passes from a clean checkout using a single command.
- (Optional) A CI run is green on push.

13. Deliverables

- A tests/ directory containing the suite.
- Test configuration (pytest.ini and/or conftest.py).
- (Optional) A CI workflow file.

14. Evaluation Criteria

Criterion	Weight
Coverage of core logic — meaningful, not vanity, coverage	35%
Edge-case handling — failure modes are tested	25%
Isolation — no shared state, no dependence on real data	20%
Repeatability — one command, consistently green	20%

WP-05 — Funnel Analytics & Statistical Reporting

1. Project Overview & Objective

Turn lead data into decision-ready insight: compute funnel conversion, time-in-stage, and source performance, then answer a real business question with a formal hypothesis test. This module applies your statistics knowledge to a genuine business problem.

2. Business Requirements

- **BR-1:** Leadership needs to know where leads drop off in the funnel.
- **BR-2:** Marketing needs to know which lead sources convert best.
- **BR-3:** Source-performance claims must be statistically defensible, not anecdotal.

3. Functional Requirements

- **FR-1:** Compute the conversion rate at each pipeline stage.
- **FR-2:** Compute the average time-in-stage.
- **FR-3:** Compute conversion by source.
- **FR-4:** Run a hypothesis test comparing the conversion of two sources.
- **FR-5:** Report the p-value and a plain-language conclusion at a stated significance level.
- **FR-6:** Produce a funnel visualisation and a source-comparison chart.

4. Expected Output

A script or notebook that outputs the funnel chart, the source comparison, and a clearly stated statistical conclusion — naming the test used, the p-value, and the interpretation.

5. Step-by-Step Implementation Plan

1. Extract lead data from the database into a dataframe.
2. Compute stage-by-stage conversion and time-in-stage.
3. Compute conversion by source.
4. Select and run an appropriate test (two-proportion z-test or chi-square).
5. Interpret the result against a significance level of 0.05.
6. Build the funnel chart and the source-comparison chart.
7. Summarise the findings in a short written conclusion.

6. Estimated Time Required

Phase	Effort
Extract & shape data	1.5 h
Funnel & stage metrics	2.5 h
Hypothesis test	2.0 h
Charts	2.0 h
Write-up	1.0 h

Estimated total: $\approx$ 9 hours (about 1 to 1.5 days)

7. Technologies to be Used

Python, pandas, NumPy, SciPy or statsmodels, Matplotlib or Plotly

8. Tools to be Used

Jupyter or VS Code notebooks, Git

9. Database Requirements

- Read-only access to the leads table.
- No schema changes.

10. API Requirements

Optional — the module may read from the WP-03 API instead of the database directly.

11. UI/UX Expectations

- Charts must be labelled and readable (titles, axes, legends).
- The written conclusion must be understandable by a non-statistician.

12. Testing Checklist

- Conversion maths is validated against a small hand-computed example.
- Funnel percentages are internally consistent (non-increasing through the stages).
- The chosen statistical test matches the data type (proportions).
- The p-value and conclusion are stated explicitly with the significance level.
- Charts include titles, axis labels, and legends.

13. Deliverables

- The analytics script or notebook.
- The generated charts (image or interactive).
- A one-page findings summary.

14. Evaluation Criteria

Criterion	Weight
Analytical correctness — metrics and test results are right	35%
Statistical reasoning — appropriate test and correct interpretation	25%
Communication — clear charts and a plain-language conclusion	25%
Reproducibility — runs end-to-end from the data	15%

WP-06 — Operations Dashboard (Streamlit)

1. Project Overview & Objective

Provide a non-technical operations view: KPIs, filters, a funnel chart, and a recent-leads table that a counselor or manager can use without engineering help. The dashboard's numbers must reconcile with the WP-05 analytics for the same filters.

2. Business Requirements

- **BR-1:** Non-technical staff need self-serve visibility into pipeline health.
- **BR-2:** Decisions should be based on a single, consistent source of numbers.

3. Functional Requirements

- **FR-1:** KPI cards for total leads, conversion percentage, and active leads by stage.
- **FR-2:** Filters for date range and source.
- **FR-3:** A funnel chart that reflects the current filters.
- **FR-4:** A table of recent leads.
- **FR-5:** Figures that match the WP-05 analytics for the same filter selection.

4. Expected Output

A locally runnable Streamlit dashboard with working filters and charts, whose figures reconcile with the analytics module.

5. Step-by-Step Implementation Plan

1. Connect the dashboard to the API or the database.
2. Build the KPI cards.
3. Add the date-range and source filters.
4. Render the funnel chart bound to the active filters.
5. Add the recent-leads table.
6. Reconcile the displayed numbers against WP-05.
7. Polish the layout and labels.

6. Estimated Time Required

Phase	Effort
Data wiring	1.5 h
KPI cards	1.5 h
Filters	2.0 h
Charts & table	2.5 h
Reconciliation & polish	1.5 h

Estimated total: ≈ 9 hours (about 1 to 1.5 days)

7. Technologies to be Used

Python, Streamlit, pandas, A charting library

8. Tools to be Used

Streamlit, Web browser, Git

9. Database Requirements

- Read access via the API or the repository.
- No writes.

10. API Requirements

May consume the WP-03 API (preferred) or read the database directly.

11. UI/UX Expectations

- A clean, single-screen layout with the most important KPIs at the top.
- Understandable to a non-technical user within roughly ten seconds.
- Labels consistent with the rest of the system, and sensible empty states.
- Responsive to filter changes without errors.

12. Testing Checklist

- Filters change the displayed data correctly.
- KPIs and the funnel match WP-05 for identical filters.
- Empty filter results show a friendly message, not an error.
- The dashboard starts with a single documented command.

13. Deliverables

- The Streamlit application.
- A short run/usage note added to the README.

14. Evaluation Criteria

Criterion	Weight
Usability — clarity for a non-technical user	30%
Correctness — numbers reconcile with the analytics	30%
Interactivity — filters work reliably	20%
Presentation — layout, labels, and empty states	20%

WP-07 — Production API Re-platform (FastAPI + PostgreSQL)

1. Project Overview & Objective

Re-platform the lead API onto a production-grade stack — FastAPI, Pydantic, and PostgreSQL — with auto-generated docs, schema validation, dependency-injected database sessions, and externalised configuration. The goal is feature parity with the Flask version on a stack suited to scale and team development.

2. Business Requirements

- **BR-1:** The product must run on a stack suitable for scale and team development.
- **BR-2:** Configuration and secrets must not be hard-coded.
- **BR-3:** API consumers benefit from typed, self-documenting contracts.

3. Functional Requirements

- **FR-1:** Re-implement all WP-03 endpoints in FastAPI.
- **FR-2:** Define request and response schemas with Pydantic.
- **FR-3:** Persist data in PostgreSQL (via SQLAlchemy or psycopg).
- **FR-4:** Provide database sessions through dependency injection.
- **FR-5:** Load configuration and secrets from the environment (.env).
- **FR-6:** Expose interactive documentation at /docs.
- **FR-7:** Port the test suite to the FastAPI application.

4. Expected Output

A FastAPI service on PostgreSQL with working Swagger docs and feature parity with the Flask API, configured entirely through environment variables.

5. Step-by-Step Implementation Plan

1. Provision a local PostgreSQL database.
2. Define Pydantic schemas for leads and request bodies.
3. Set up the data layer (models, session management, schema initialisation/migration).
4. Implement the endpoints with dependency-injected sessions.
5. Externalise configuration via a .env file.
6. Verify the /docs page and the validation behaviour.
7. Port and run the test suite against the new stack.

6. Estimated Time Required

Phase	Effort
PostgreSQL setup	1.5 h
Pydantic schemas	1.5 h
Data layer	3.0 h
Endpoints	3.0 h
Config & docs	1.5 h
Port tests	2.5 h

Estimated total: ≈ 13 hours (about 2 days)

7. Technologies to be Used

Python, FastAPI, Pydantic, PostgreSQL, SQLAlchemy or psycopg, Uvicorn

8. Tools to be Used

PostgreSQL (local), pgAdmin or psql, Postman, python-dotenv, Git

9. Database Requirements

- A PostgreSQL database with a leads table mirroring the WP-01 schema, using typed columns, constraints, and indexes on phone and stage.
- Connection parameters supplied via environment variables, never committed.
- An initialisation or migration script for schema creation.

10. API Requirements

Method	Endpoint	Purpose	Status codes
GET	/api/v1/leads	List leads (same query params as WP-03)	200
GET	/api/v1/leads/{id}	Retrieve a single lead	200 / 404
POST	/api/v1/leads	Create a lead (Pydantic-validated)	201 / 422
PATCH	/api/v1/leads/{id}/stage	Move a lead to a new stage	200 / 404 / 422
DELETE	/api/v1/leads/{id}	Delete a lead	204 / 404

Notes: Same contract as WP-03, served by FastAPI with Pydantic validation and an auto-generated OpenAPI document at /docs.

11. UI/UX Expectations

- No GUI beyond the interactive Swagger docs page at /docs, which must render and allow try-it-out calls.

12. Testing Checklist

- All endpoints reach parity with the Flask version.
- Invalid payloads are rejected by Pydantic with clear messages.
- /docs renders and reflects the defined schemas.
- No credentials appear in the code or version history.
- The ported test suite passes against PostgreSQL.

13. Deliverables

- The FastAPI application and data layer.
- A .env.example documenting the required variables.
- A schema/migration script.
- The updated test suite.

14. Evaluation Criteria

Criterion	Weight
Feature parity — matches the Flask behaviour	30%
Stack correctness — proper FastAPI / Pydantic / PostgreSQL usage	25%

Criterion	Weight
Configuration hygiene — env-based, no secrets committed	20%
Validation & docs — Pydantic models and a working /docs page	15%
Tests — the suite passes on the new stack	10%

WP-08 — Lead Intent Classification & Automation

1. Project Overview & Objective

Add a lightweight intelligence layer that classifies inbound lead messages by intent and suggests the next stage or a templated reply, exposed as an API endpoint. This mirrors an assisted-counselor workflow: triaging repetitive inbound questions automatically.

2. Business Requirements

- **BR-1:** Counselors spend significant time triaging repetitive inbound questions.
- **BR-2:** Routing leads to the right next step faster improves conversion.

3. Functional Requirements

- **FR-1:** Accept an inbound message tied to a specific lead.
- **FR-2:** Classify the message into an intent (for example: fees, timing, eligibility, not-interested, other).
- **FR-3:** Suggest a next stage and/or a templated reply.
- **FR-4:** Expose this through POST /api/v1/leads/{id}/message.
- **FR-5:** Provide a way to measure classification accuracy on a labelled sample.

4. Expected Output

An endpoint that, given a message, returns an intent, a suggested next action, and optionally a reply — together with a small accuracy evaluation on a labelled set.

5. Step-by-Step Implementation Plan

1. Define the intent taxonomy and assemble a labelled test set.
2. Implement a rule-based classifier (keywords and patterns).
3. Map intents to suggested stages and replies.
4. Expose the message endpoint.
5. Evaluate accuracy against the labelled set.
6. (Stretch) Swap in an LLM-based classifier and compare accuracy and latency.

6. Estimated Time Required

Phase	Effort
Taxonomy & labelled data	1.5 h
Rule-based classifier	2.5 h
Mapping & endpoint	2.0 h
Accuracy evaluation	1.5 h
LLM comparison (stretch)	2.0 h

Estimated total: ≈ 7.5 hours core (about 1 day), plus ~2 hours for the stretch goal

7. Technologies to be Used

Python, FastAPI, (stretch) a local or hosted LLM

8. Tools to be Used

Postman, Git, (stretch) an Ollama or LLM API client

9. Database Requirements

- May optionally persist messages and their predicted intents in a new table; otherwise the feature is stateless.

10. API Requirements

Method	Endpoint	Purpose	Status codes
POST	/api/v1/leads/{id}/message	Classify an inbound message and suggest a next action	200 / 400 / 404

Notes: Request body: {"message": "..."}.
Response: {"intent": "...", "suggested_stage": "...", "reply": "..."} (optional).

11. UI/UX Expectations

- No GUI — the JSON contract is the interface. (Stretch: surface suggestions inside the WP-06 dashboard.)

12. Testing Checklist

- Representative messages map to the correct intent.
- Unknown or ambiguous messages fall back to “other” gracefully.
- The endpoint returns 404 for an unknown lead id.
- Accuracy is measured and reported on the labelled set.
- (Stretch) The rule-based vs LLM comparison is documented.

13. Deliverables

- The classifier module and endpoint.
- A labelled evaluation set and an accuracy report.
- (Stretch) LLM comparison notes.

14. Evaluation Criteria

Criterion	Weight
Functional accuracy — correct, useful classifications	30%
Robustness — sensible fallback, no crashes	25%
Evaluation rigor — measured accuracy and honest limits	25%
Integration — a clean endpoint consistent with the API	20%

WP-09 — System Integration & Release (Capstone)

1. Project Overview & Objective

Integrate all modules into one coherent, documented, deployable product — “AcademyOps v1” — that a new engineer can clone and run from the README alone. This capstone simulates a real handover at the end of an engagement.

2. Business Requirements

- **BR-1:** The client needs a single, runnable system, not a set of disconnected parts.
- **BR-2:** The system must be handed over with documentation sufficient for onboarding.

3. Functional Requirements

- **FR-1:** Integrate the FastAPI + PostgreSQL backend, the analytics, the dashboard, and the classifier.
- **FR-2:** Provide complete setup-and-run documentation.
- **FR-3:** Provide an architecture overview (diagram plus description).
- **FR-4:** Ensure the full automated test suite passes.
- **FR-5:** (Stretch) Containerise the system with Docker and docker-compose.

4. Expected Output

A single repository where, by following only the README, another engineer can stand up the backend, the dashboard, and the analytics, and run the tests successfully.

5. Step-by-Step Implementation Plan

1. Consolidate the modules and resolve any integration gaps.
2. Finalise the configuration and run scripts.
3. Write end-to-end setup, run, and test documentation.
4. Produce the architecture diagram and its description.
5. Run the full suite and perform a clean-clone walkthrough.
6. (Stretch) Add a Dockerfile and docker-compose for the app and the database.

6. Estimated Time Required

Phase	Effort
Integration	3.0 h
Documentation	2.5 h
Architecture diagram	1.5 h
Clean-clone verification	2.0 h
Docker (stretch)	3.0 h

Estimated total: ≈ 9 hours core (about 1.5 days), plus ~3 hours for the stretch goal

7. Technologies to be Used

All prior technologies, (stretch) Docker and docker-compose

8. Tools to be Used

Git, A diagram tool (draw.io or Excalidraw), Docker (stretch)

9. Database Requirements

- The PostgreSQL database from WP-07, provisioned via documented steps or via docker-compose.

10. API Requirements

The consolidated FastAPI service from WP-07 and WP-08.

11. UI/UX Expectations

- The WP-06 dashboard, plus a readable README and architecture diagram acting as the handover interface.

12. Testing Checklist

- A clean clone plus the README steps brings up the whole system.
- The full automated test suite passes.
- The dashboard, the analytics, and the API all run and agree on the numbers.
- The architecture diagram matches the actual components.
- (Stretch) docker-compose up brings up the app and the database together.

13. Deliverables

- The integrated repository, tagged v1.0.
- A complete README and architecture diagram.
- A passing test suite.
- (Stretch) Docker assets.

14. Evaluation Criteria

Criterion	Weight
Integration completeness — everything works together	30%
Onboarding quality — a clean clone runs from the docs alone	25%
Documentation & architecture — clear and accurate	25%
Reliability — tests green and numbers consistent	20%